

Programming Fundamentals in C

Chapter 9: Formatted Input Output

Umer Shahid

Department of Electrical Engineering, UET Lahore

December 10, 2025

Today's Agenda

1. Introduction
2. Understanding Streams
3. Formatting Output with printf
4. Printing Integers
5. Printing Floating-Point Numbers
6. Printing Strings and Characters
7. Other Conversion Specifiers
8. Field Widths and Precision
9. printf Format Flags
10. Printing Literals and Escape Sequences
11. Formatted Input with scanf
12. Practical Examples and Case Studies

Chapter Overview

What Will You Learn?

- How to format output using `printf`
- How to format input using `scanf`
- Printing integers, floating-point numbers, strings, and characters
- Using field widths and precisions
- Special formatting flags
- Reading formatted input safely

Important Note

This chapter focuses on presenting results beautifully and reading input correctly - essential skills for any programmer!

Why Formatting Matters

Without Formatting:

```
1 Price: 5.5
2 Total: 123.456789
```

With Formatting:

```
1 Price: $ 5.50
2 Total: $ 123.46
```

Professional Output

Formatting makes your programs look professional and user-friendly!

What are Streams?

Definition

A **stream** is a sequence of bytes that flows between your program and devices (keyboard, screen, files, etc.)

Input Stream:

- Bytes flow INTO memory
- From: keyboard, files, network
- Direction: Device → Program

Output Stream:

- Bytes flow OUT OF memory
- To: screen, printer, files
- Direction: Program → Device

Three Standard Streams

Standard Input Stream

- Connected to: **Keyboard**
- Used for: Reading user input
- Function: `scanf()`

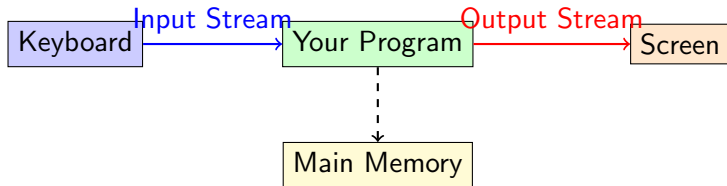
Standard Output Stream

- Connected to: **Screen**
- Used for: Displaying normal output
- Function: `printf()`

Standard Error Stream

- Connected to: **Screen**
- Used for: Displaying error messages
- Function: `fprintf(stderr, ...)`

Visual Representation of Streams



Self-Check: Streams

Fill in the Blank

You can _____ the standard streams to other devices.

Self-Check: Streams

Fill in the Blank

You can _____ the standard streams to other devices.

Answer

redirect

Self-Check: Streams

Fill in the Blank

You can _____ the standard streams to other devices.

Answer

redirect

Multiple Choice

Which statement is FALSE?

- a) Input and output are performed with arrays
- b) In input operations, bytes flow from device to memory
- c) In output operations, bytes flow from memory to device
- d) Standard streams connect when execution begins

Self-Check: Streams

Fill in the Blank

You can _____ the standard streams to other devices.

Answer

redirect

Multiple Choice

Which statement is FALSE?

- a) Input and output are performed with arrays
- b) In input operations, bytes flow from device to memory
- c) In output operations, bytes flow from memory to device
- d) Standard streams connect when execution begins

Answer

a) is false - Actually performed with **streams**, not arrays!

The printf Function

Basic Syntax

```
1 printf(format-control-string, other-arguments);
```

Required Header

```
1 #include <stdio.h>
```

Components of Format Control String

- **Conversion specifiers** - Tell how to format data
- **Flags** - Modify the output format
- **Field widths** - Specify output width
- **Precisions** - Control decimal places
- **Literal characters** - Printed as-is

printf Capabilities

1. **Round** floating-point values to decimal places
2. **Align** columns of numbers at decimal points
3. **Right-align** and **left-align** outputs
4. **Insert** literal characters at precise locations
5. Display floating-point in **exponential format**
6. Display unsigned integers in **octal** and **hexadecimal**
7. Display data with **fixed-size field widths**

Conversion Specifications

Structure

A conversion specification always:

- **Begins** with % (percent sign)
- **Ends** with a conversion specifier character

Examples

```
1 printf("Integer: %d\n", 42);  
2 printf("Float: %f\n", 3.14);  
3 printf("Character: %c\n", 'A');  
4 printf("String: %s\n", "Hello");
```

Important

You can have MANY conversion specifications in ONE format control string!

Self-Check: printf Basics

Fill in the Blank

Every printf call contains a _____ that describes the output format.

Self-Check: printf Basics

Fill in the Blank

Every printf call contains a _____ that describes the output format.

Answer

format control string

Self-Check: printf Basics

Fill in the Blank

Every printf call contains a _____ that describes the output format.

Answer

format control string

Multiple Choice

Which is a printf formatting capability?

- a) Rounding floating-point values
- b) Representing floating-point in exponential format
- c) Displaying data with fixed-size field widths
- d) All of the above

Self-Check: printf Basics

Fill in the Blank

Every printf call contains a _____ that describes the output format.

Answer

format control string

Multiple Choice

Which is a printf formatting capability?

- a) Rounding floating-point values
- b) Representing floating-point in exponential format
- c) Displaying data with fixed-size field widths
- d) All of the above

Answer

d) All of the above

Integer Conversion Specifiers

Specifier	Description
%d	Display as signed decimal integer
%i	Display as signed decimal integer (same as %d)
%o	Display as unsigned octal integer
%u	Display as unsigned decimal integer
%x	Display as unsigned hexadecimal (lowercase a-f)
%X	Display as unsigned hexadecimal (uppercase A-F)
%hd	Display short integer
%ld	Display long integer
%lld	Display long long integer

Integer Examples - Part 1

```
1 #include <stdio.h>
2
3 int main(void) {
4     printf("%d\n", 455);           // Output: 455
5     printf("%i\n", 455);           // Output: 455
6     printf("%d\n", +455);          // Output: 455 (+ doesn't print)
7     printf("%d\n", -455);          // Output: -455
8 }
```

Key Point

- %d and %i are identical for printf
- Plus sign doesn't display by default
- Minus sign always displays

Integer Examples - Part 2

```
1 #include <stdio.h>
2
3 int main(void) {
4     printf("%hd\n", 32000);           // short: 32000
5     printf("%o\n", 455);             // octal: 707
6     printf("%u\n", 455);             // unsigned: 455
7     printf("%x\n", 455);             // hex: 1c7
8     printf("%X\n", 455);             // HEX: 1C7
9 }
```

Common Error: Unsigned with Negative

```
1 printf("%u\n", -455);
```

Output: 4294966841

Logic Error!

- Printing a negative value with %u is WRONG
- The negative value displays as a large positive integer
- This happens due to the "sign bit" in binary representation
- Always use %d for signed integers!

Correct Usage

```
1 printf("%d\n", -455); // Output: -455
```

Number Systems Quick Reference

Decimal	Octal	Hexadecimal	Binary
10	12	A	1010
15	17	F	1111
455	707	1C7	111000111

Remember

- **Octal:** Base 8 (digits 0-7)
- **Decimal:** Base 10 (digits 0-9)
- **Hexadecimal:** Base 16 (digits 0-9, A-F)

Self-Check: Printing Integers

Multiple Choice

Which specifier means "Display as unsigned decimal integer"?

- a) %ud
- b) %ui
- c) %u
- d) None of the above

Self-Check: Printing Integers

Multiple Choice

Which specifier means "Display as unsigned decimal integer"?

- a) %ud
- b) %ui
- c) %u
- d) None of the above

Answer

c) %u

Floating-Point Conversion Specifiers

Specifier	Description
%e	Display in exponential notation (lowercase e)
%E	Display in exponential notation (uppercase E)
%f	Display in fixed-point notation
%F	Display in fixed-point notation (uppercase)
%g	Display in e or f format (automatic, lowercase)
%G	Display in E or F format (automatic, uppercase)
%Lf	Display long double

Default Precision

All floating-point specifiers show **6 digits** after the decimal point by default.

Understanding Exponential Notation

Scientific Notation (Mathematics)

$$150.4582 = 1.504582 \times 10^2$$

Exponential Notation (Computer)

1.504582E+02

Components

- **E** = "exponent"
- **1.504582** = mantissa (significand)
- **+02** = exponent (power of 10)
- Meaning: $1.504582 \times 10^{+2}$

Specifiers e, E, and f

```
1 #include <stdio.h>
2
3 int main(void) {
4     printf("%e\n", 1234567.89);    // 1.234568e+06
5     printf("%E\n", 1234567.89);    // 1.234568E+06
6     printf("%f\n", 1234567.89);    // 1234567.890000
7 }
```

Key Differences

- %e/%E: Shows exactly 1 digit before decimal point
- %f/%F: Shows at least 1 digit before decimal point
- %e uses lowercase 'e', %E uses uppercase 'E'
- All show 6 decimal places by default

Specifiers g and G (Automatic Selection)

```
1 printf("%g\n", 1234567.89);    // 1.23457e+06
2 printf("%G\n", 1234567.89);    // 1.23457E+06
```

How g/G Decides

Uses e/E format if:

- Exponent < -4 , OR
- Exponent $\geq$ precision

Otherwise, uses f format

Special Features

- Removes trailing zeros (1.234000 $\rightarrow$ 1.234)
- Default precision: 6 significant digits
- Chooses most compact representation

g/G Examples with Different Values

```
1 printf("%g\n", 0.0000875);      // 8.75e-05   (exponent < -4)
2 printf("%g\n", 8750000.0);      // 8.75e+06   (exponent >= 6)
3 printf("%g\n", 8.75);           // 8.75       (normal range)
4 printf("%g\n", 87.50);          // 87.5       (trailing 0 removed)
```

Why These Formats?

- 0.0000875: Exponent is -5, which is < -4 → use e format
- 8750000.0: Exponent is 6, which equals precision → use e format
- 8.75: Normal range → use f format
- 87.50: Trailing zero removed → displays as 87.5

Complete Floating-Point Example

```
1 #include <stdio.h>
2
3 int main(void) {
4     printf("%e\n", 1234567.89);    // 1.234568e+06
5     printf("%e\n", +1234567.89);   // 1.234568e+06 (+ doesn't print)
6     printf("%e\n", -1234567.89);   // -1.234568e+06
7     printf("%E\n", 1234567.89);    // 1.234568E+06
8     printf("%f\n", 1234567.89);    // 1234567.890000
9     printf("%g\n", 1234567.89);    // 1.23457e+06
10    printf("%G\n", 1234567.89);    // 1.23457E+06
11 }
```

Note

%E, %e, and %g perform rounding, but %f does not in this example.

Self-Check: Floating-Point

Fill in the Blank

The conversion specifiers e and E display floating-point values in exponential notation—the computer equivalent of _____ used in mathematics.

Self-Check: Floating-Point

Fill in the Blank

The conversion specifiers e and E display floating-point values in exponential notation—the computer equivalent of _____ used in mathematics.

Answer

scientific notation

Character and String Specifiers

Specifier	Description
<code>%c</code>	Print a single character
<code>%s</code>	Print a string (until 0)

Requirements

- `%c`: Requires a `char` argument
- `%s`: Requires a pointer to `char` (string)

Important!

`%s` prints until it finds a null character (`0`). If there's no null terminator, the program may crash!

Character and String Examples

```
1 #include <stdio.h>
2
3 int main(void) {
4     char character = 'A';
5     printf("%c\n", character);           // A
6
7     printf("%s\n", "This is a string"); // This is a string
8
9     char string[] = "This is a string";
10    printf("%s\n", string);              // This is a string
11
12    const char *stringPtr = "This is also a string";
13    printf("%s\n", stringPtr);           // This is also a string
14 }
```

Common Errors with Characters and Strings

Error 1: Using %c for Strings

```
1 printf("%c\n", "Hello"); // WRONG! Logic Error
```

- %c expects a char, not a string
- A string is a pointer to char (char *)

Error 2: Using %s for Characters

```
1 printf("%s\n", 'A'); // FATAL ERROR!
```

- %s expects a pointer to char
- Treats the character's numeric value as a memory address
- Usually causes "access violation" or "segmentation fault"

Understanding the Null Terminator

What is 0?

- A special character marking the end of a string
- ASCII value is 0
- Not visible, but MUST be present

```
1 char name[] = "Ali";  
2 // Actually stored as: 'A', 'l', 'i', '\0'
```

Without Null Terminator

```
1 char bad[3] = {'A', 'l', 'i'}; // No \0!  
2 printf("%s", bad); // May print: Ali?????junk?????  
3 // Or crash with segmentation fault!
```

Format Control String Errors

Compilers Don't Catch These Errors!

Most format control string errors only appear at:

- Runtime (when program executes)
- As incorrect results
- As program crashes

Always Double-Check

- Match conversion specifiers with argument types
- %d for int
- %f for float/double
- %c for char
- %s for string (char *)

Self-Check: Strings and Characters

Fill in the Blank

Conversion specifier `s` causes characters to be printed until a _____ is encountered.

Self-Check: Strings and Characters

Fill in the Blank

Conversion specifier s causes characters to be printed until a _____ is encountered.

Answer

terminating null (0) character

Self-Check: Strings and Characters

Fill in the Blank

Conversion specifier `s` causes characters to be printed until a _____ is encountered.

Answer

terminating null (`0`) character

True/False

Compilers catch errors in the format-control string, so you won't experience incorrect results at runtime.

Self-Check: Strings and Characters

Fill in the Blank

Conversion specifier `s` causes characters to be printed until a _____ is encountered.

Answer

terminating null (`0`) character

True/False

Compilers catch errors in the format-control string, so you won't experience incorrect results at runtime.

Answer

FALSE! Most compilers do NOT catch format-control string errors. You'll only discover them at runtime when your program fails or produces wrong output!

The p and % Specifiers

Specifier	Description
%p	Display a pointer value (memory address)
%%	Display the percent character

Usage

```
1 int x = 12345;
2 int *ptr = &x;
3
4 printf("The value of ptr is %p\n", ptr);
5 printf("The address of x is %p\n", &x);
6 printf("Printing a %% in format string\n");
```

Pointer Display Example

```
1 #include <stdio.h>
2
3 int main(void) {
4     int x = 12345;
5     int *ptr = &x;
6
7     printf("The value of ptr is %p\n", ptr);
8     printf("The address of x is %p\n\n", &x);
9
10    printf("Printing a %% in a format control string\n");
11 }
```

Output:

The value of ptr is 0x7ffff6eb911c

The address of x is 0x7ffff6eb911c

Printing a % in a format control string

Understanding Pointer Display

Why Are They The Same?

```
1 int *ptr = &x; // ptr stores the address of x
```

Both `ptr` and `&x` have identical values because `ptr` contains `x`'s address!

Implementation-Defined

- `%p` format is implementation-defined
- Usually displays in hexadecimal notation
- Your addresses will be different each time!
- Memory addresses change between program executions

Why %% for Percent Sign?

The Problem

```
1 printf("Complete: 75%\n"); // ERROR!
```

- % signals start of conversion specification
- % n is not a valid specifier
- Program expects a conversion specifier after %

The Solution

```
1 printf("Complete: 75%%\n"); // Output: Complete: 75%
```

Use %% to print a literal percent character!

Self-Check: Other Specifiers

Fill in the Blank

A printf statement uses _____ to print the % character.

Self-Check: Other Specifiers

Fill in the Blank

A printf statement uses _____ to print the % character.

Answer

%%

Self-Check: Other Specifiers

Fill in the Blank

A printf statement uses _____ to print the % character.

Answer

%%

True/False

The conversion specifier p displays an address in decimal notation.

Self-Check: Other Specifiers

Fill in the Blank

A printf statement uses _____ to print the % character.

Answer

%%

True/False

The conversion specifier p displays an address in decimal notation.

Answer

FALSE! The conversion specifier p displays an address in an implementation-defined manner—typically using hexadecimal notation.

What is Field Width?

Definition

Field width specifies the exact size of the field in which data is printed.

Behavior

- If field width $>$ data size: Data is right-aligned
- If field width $<$ data size: Data prints in full anyway
- Negative values use one position for minus sign

Syntax

Insert integer between % and conversion specifier:

`%4d` `%10s` `%8f`

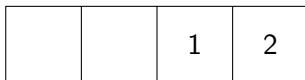
Field Width Example - Integers

```
1 #include <stdio.h>
2
3 int main(void) {
4     printf("%4d\n", 1);           //    1
5     printf("%4d\n", 12);          //   12
6     printf("%4d\n", 123);         //  123
7     printf("%4d\n", 1234);        // 1234
8     printf("%4d\n\n", 12345);     // 12345 (exceeds width!)
9
10    printf("%4d\n", -1);           //   -1
11    printf("%4d\n", -12);          //  -12
12    printf("%4d\n", -123);         // -123
13    printf("%4d\n", -1234);        // -1234
14    printf("%4d\n", -12345);       // -12345
15 }
```

Visual Representation of Field Width

```
printf("%4d", 12);
```

Right-aligned



← 4 characters wide →

Important

If the value is wider than the field, it prints in full - your columns may misalign! Always know your data!

Understanding Precision

Precision Means Different Things

For Integers:

- Minimum number of digits to print
- Pads with leading zeros if needed
- Default precision: 1

For Floating-Point (e, E, f):

- Number of digits AFTER decimal point
- Default precision: 6

For g and G:

- Maximum number of significant digits
- Default precision: 6

For Strings (s):

- Maximum number of characters to print

Precision Syntax

Format

Place a decimal point (.) followed by an integer between % and specifier:

```
1 printf("%.4d", 873);           // Integer precision
2 printf("%.3f", 123.945);       // Float precision
3 printf("%.11s", "Hello");      // String precision
```

Examples

- %.4d: Print integer with minimum 4 digits
- %.3f: Print float with 3 decimal places
- %.11s: Print maximum 11 characters of string

Precision Examples - Complete Program

```
1 #include <stdio.h>
2
3 int main(void) {
4     puts("Using precision for integers");
5     int i = 873;
6     printf("\t%.4d\n\t%.9d\n\n", i, i);
7
8     puts("Using precision for floating-point numbers");
9     double f = 123.94536;
10    printf("\t%.3f\n\t%.3e\n\t%.3g\n\n", f, f, f);
11
12    puts("Using precision for strings");
13    char s[] = "Happy Birthday";
14    printf("\t%.11s\n", s);
15 }
```

Precision Examples - Output

Using precision for integers

0873

000000873

Using precision for floating-point numbers

123.945

1.239e+02

124

Using precision for strings

Happy Birth

Observations

- Integers padded with leading zeros
- Floats rounded to specified decimal places
- String truncated to 11 characters

Combining Field Width and Precision

Syntax

`%width.precision specifier`

```
1 printf ("%9.3f", 123.456789);
```

Breakdown

- 9: Total field width (including decimal point)
- .3: Three digits after decimal point
- Result: 123.457 (right-aligned in 9 spaces)

		1	2	3	.	4	5	7
--	--	---	---	---	---	---	---	---

Dynamic Width and Precision with *

Using Asterisk (*)

You can specify width and precision using variables!

```
1 printf("%*.*f", 7, 2, 98.736);
```

How It Works

- First * (7): Field width = 7
- Second * (2): Precision = 2
- Third argument (98.736): The value to print
- Result: 98.74 (right-aligned)

Useful For

When width or precision changes based on program logic!

```
1 int width = 10;  
2 int precision = 3;
```

Self-Check: Field Widths and Precision

Multiple Choice

Which statement is FALSE?

- a) Default precision for integers is 1
- b) For e, E, f: precision is digits BEFORE decimal point
- c) For g, G: precision is max significant digits
- d) For s: precision is max characters to print

Self-Check: Field Widths and Precision

Multiple Choice

Which statement is FALSE?

- a) Default precision for integers is 1
- b) For e, E, f: precision is digits BEFORE decimal point
- c) For g, G: precision is max significant digits
- d) For s: precision is max characters to print

Answer

b) is FALSE! For e, E, and f, precision is the number of digits AFTER the decimal point.

Self-Check: Field Widths and Precision

Multiple Choice

Which statement is FALSE?

- a) Default precision for integers is 1
- b) For e, E, f: precision is digits BEFORE decimal point
- c) For g, G: precision is max significant digits
- d) For s: precision is max characters to print

Answer

b) is FALSE! For e, E, and f, precision is the number of digits AFTER the decimal point.

What Does This Print?

```
printf("%9.3f", 123.456789);
```

Self-Check: Field Widths and Precision

Multiple Choice

Which statement is FALSE?

- a) Default precision for integers is 1
- b) For e, E, f: precision is digits BEFORE decimal point
- c) For g, G: precision is max significant digits
- d) For s: precision is max characters to print

Answer

b) is FALSE! For e, E, and f, precision is the number of digits AFTER the decimal point.

What Does This Print?

```
printf("%9.3f", 123.456789);
```

Answer

Right-aligns in a 9-digit field the rounded value **123.457**

Format Flags Overview

Flag	Description
-	Left-align output within field
+	Display + for positive, - for negative
space	Print space before positive values
#	Prefix 0 for octal, 0x for hex; force decimal point
0	Pad field with leading zeros

Flag Placement

Flags go immediately after % and before conversion specifier:

`%-10d` `%+09d` `%#x`

Multiple Flags

You can combine several flags: `%+09.3f`

Flag: - (Minus Sign) - Left Alignment

```
1 #include <stdio.h>
2
3 int main(void) {
4     puts("1234567890123456789012345678901234567890");
5     printf("%10s%10d%10c%10f\n\n",
6           "hello", 7, 'a', 1.23);
7
8     puts("1234567890123456789012345678901234567890");
9     printf("%-10s%-10d%-10c%-10f\n",
10          "hello", 7, 'a', 1.23);
11 }
```

Output:

```
1234567890123456789012345678901234567890
      hello          7          a  1.230000
```

```
1234567890123456789012345678901234567890
hello      7          a          1.230000
```

Visual: Right vs Left Alignment

Right-Aligned (default): %10d

Right-aligned

							1	2	3
--	--	--	--	--	--	--	---	---	---

Left-Aligned: %-10d

Left-aligned

1	2	3							
---	---	---	--	--	--	--	--	--	--

Flag: + (Plus Sign)

```
1 #include <stdio.h>
2
3 int main(void) {
4     printf("%d\n%d\n", 786, -786);
5     printf("%+d\n%+d\n", 786, -786);
6 }
```

Output:

786
-786
+786
-786

Key Points

- Without +: Only minus sign shows for negatives
- With +: Plus sign shows for positives, minus for negatives
- Useful for aligned positive/negative columns

Flag: space (Space Flag)

```
1 #include <stdio.h>
2
3 int main(void) {
4     printf("% d\n% d\n", 547, -547);
5 }
```

Output:

547
-547

Purpose

- Adds space before positive numbers
- Negative numbers still show minus sign
- Aligns positive and negative values with same digits
- Useful for column alignment in tables

Flag: # (Hash/Pound Sign)

```
1 #include <stdio.h>
2
3 int main(void) {
4     int c = 1427;
5     printf("%#o\n", c);        // 02623
6     printf("%#x\n", c);        // 0x593
7     printf("%#X\n", c);        // 0X593
8
9     double p = 1427.0;
10    printf("\n%g\n", p);        // 1427
11    printf("%#g\n", p);        // 1427.00
12 }
```

Effects

- `%#o`: Prefixes 0 to octal
- `%#x`: Prefixes 0x to hexadecimal
- `%#X`: Prefixes 0X to hexadecimal
- `%#g`: Forces decimal point for n/C

Flag: 0 (Zero Flag)

```
1 #include <stdio.h>
2
3 int main(void) {
4     printf("%+09d\n", 452);    // +00000452
5     printf("%09d\n", 452);    // 000000452
6 }
```

Purpose

- Pads field with leading zeros instead of spaces
- Often combined with + flag
- Useful for fixed-width numeric displays
- Example: Serial numbers, IDs, dates

Comparison

```
1 printf("%9d\n", 452);    //      452 (spaces)
2 printf("%09d\n", 452);    // 000000452 (zeros)
```

Self-Check: Format Flags

Multiple Choice

Which printf format flag displays + for positive and - for negative?

- a) -
- b) +
- c) 0
- d) None

Self-Check: Format Flags

Multiple Choice

Which printf format flag displays + for positive and - for negative?

- a) -
- b) +
- c) 0
- d) None

Answer

b) +

Self-Check: Format Flags

Multiple Choice

Which printf format flag displays + for positive and - for negative?

- a) -
- b) +
- c) 0
- d) None

Answer

b) +

What Does This Print?

```
1 printf("%d\n%d\n", 437, -437);  
2 printf("%+d\n%+d\n", 437, -437);
```

Self-Check: Format Flags

Multiple Choice

Which printf format flag displays + for positive and - for negative?

- a) -
- b) +
- c) 0
- d) None

Answer

b) +

What Does This Print?

```
1 printf("%d\n%d\n", 437, -437);  
2 printf("%+d\n%+d\n", 437, -437);
```

Escape Sequences Table

Escape Sequence	Description
'	Output single quote (')
"	Output double quote (")
?	Output question mark (?)
	Output backslash (\)
a	Alert (bell) or visual alert
b	Backspace (move cursor back one)
f	Form feed (new page)
n	Newline (move to next line)
r	Carriage return (move to line start)
t	Horizontal tab
v	Vertical tab

Why Escape Sequences?

Problem Characters

Some characters can't be typed directly in strings:

- **Newline:** Can't press Enter in a string
- **Tab:** Would indent your code oddly
- **Quote marks:** Would end the string early

Solution: Escape Sequences

```
1 printf("Line 1\nLine 2\n");           // Newline
2 printf("Name:\tJohn\n");             // Tab
3 printf("He said \"Hello\"\n");        // Quote marks
4 printf("Path: C:\\Users\\Ali\n");     // Backslash
```

Common Escape Sequences Examples

```
1 #include <stdio.h>
2
3 int main(void) {
4     printf("Hello\nWorld\n");           // Newline
5     printf("Name:\tAge\n");             // Tab
6     printf("John\t25\n");
7     printf("He said \"Hi\"\n");         // Quotes
8     printf("Path: C:\\Program Files\n"); // Backslash
9     printf("100%% Complete\n");         // Percent
10    printf("\a");                        // Alert/Beep
11 }
```

Output:

Hello

World

Name: Age

John 25

He said "Hi"

Practical Escape Sequence Uses

Creating Tables

```
1 printf("Name\tAge\tCity\n");  
2 printf("Ali\t20\tLahore\n");  
3 printf("Sara\t19\tKarachi\n");
```

Output:

Name	Age	City
Ali	20	Lahore
Sara	19	Karachi

Displaying Paths

```
1 printf("Save file: \"C:\\Users\\Documents\\file.txt\"\\n");
```

Output:

Save file: "C:\Users\Documents\file.txt"

scanf Overview

Definition

scanf reads formatted input from the standard input stream (keyboard).

Capabilities

1. Input all types of data
2. Input specific characters from stream
3. Skip specific characters in stream

Syntax

```
1 scanf (format-control-string , other-arguments);
```

Key Difference from printf

Arguments to scanf must be **POINTERS** (addresses) - use & operator!

scanf Conversion Specifiers - Integers

Specifier	Description
%d	Read signed decimal integer
%i	Read decimal, octal, OR hexadecimal
%o	Read octal integer
%u	Read unsigned decimal integer
%x, %X	Read hexadecimal integer
%hd	Read short integer
%ld	Read long integer
%lld	Read long long integer

Important!

%d and %i are DIFFERENT for scanf! %i can read octal and hex!

scanf Conversion Specifiers - Floating-Point

Specifier	Description
<code>%e</code> , <code>%E</code>	Read floating-point (exponential)
<code>%f</code>	Read floating-point (fixed)
<code>%g</code> , <code>%G</code>	Read floating-point (any format)
<code>%lf</code>	Read double
<code>%Lf</code>	Read long double

Critical!

For double, you MUST use `%lf` in `scanf` (but `%f` works in `printf`)!

scanf Conversion Specifiers - Chars & Strings

Specifier	Description
%c	Read single character (no null added)
%s	Read string (null automatically added)
%[...]	Scan set - read specific characters
%p	Read pointer (address)
%n	Store number of chars read so far
%%	Skip percent sign in input

Case Study 1: Formatted Report

```
1 #include <stdio.h>
2
3 int main(void) {
4     printf("%-15s%10s%10s\n", "Product", "Price", "Qty");
5     printf("%-15s%10s%10s\n", "-----", "-----", "----");
6
7     printf("%-15s%10.2f%10d\n", "Laptop", 45000.50, 5);
8     printf("%-15s%10.2f%10d\n", "Mouse", 850.00, 25);
9     printf("%-15s%10.2f%10d\n", "Keyboard", 2500.75, 15);
10
11     printf("%-15s%10s%10s\n", "-----", "-----", "----");
12     printf("%-15s%10.2f%10d\n", "Total", 48351.25, 45);
13 }
```

Case Study 1: Output

Product	Price	Qty
-----	-----	---
Laptop	45000.50	5
Mouse	850.00	25
Keyboard	2500.75	15
-----	-----	---
Total	48351.25	45

Techniques Used

- %-15s: Left-align in 15 spaces
- %10.2f: Right-align with 2 decimals
- %10d: Right-align integer
- Creates professional-looking table

Case Study 2: Student Grade System

```
1 int main(void) {
2     char name[50];
3     int rollNo;
4     float marks1, marks2, marks3;
5     printf("Enter student name: ");
6     scanf("%49s", name);
7     printf("Enter roll number: ");
8     scanf("%d", &rollNo);
9     printf("Enter three subject marks: ");
10    scanf("%f%f%f", &marks1, &marks2, &marks3);
11    float average = (marks1 + marks2 + marks3) / 3.0;
12    printf("\n%-20s: %s\n", "Student Name", name);
13    printf("%-20s: %d\n", "Roll Number", rollNo);
14    printf("%-20s: %.2f\n", "Average Marks", average);
15 }
```

Case Study 2: Sample Run

Input:

Enter student name: Ahmed

Enter roll number: 12345

Enter three subject marks: 85.5 90.0 78.5

Output:

Student Name : Ahmed

Roll Number : 12345

Average Marks : 84.67

Learning Points

- Safe string input with field width
- Multiple scanf calls for clarity
- Formatted output with labels
- Floating-point precision control

Case Study 3: Time Formatting

```
1 #include <stdio.h>
2 int main(void) {
3     int hours, minutes, seconds;
4     printf("Enter time (HH:MM:SS): ");
5     scanf("%d:%d:%d", &hours, &minutes, &seconds);
6     // Display in different formats
7     printf("\n24-hour format: %02d:%02d:%02d\n",
8           hours, minutes, seconds);
9     // Convert to 12-hour format
10    int displayHour = hours > 12 ? hours - 12 : hours;
11    if (displayHour == 0) displayHour = 12;
12    printf("12-hour format: %02d:%02d:%02d %s\n",
13          displayHour, minutes, seconds,
14          hours >= 12 ? "PM" : "AM");
15 }
```

Case Study 3: Sample Run

Input: 14:05:30

Output:

24-hour format: 14:05:30

12-hour format: 02:05:30 PM

Techniques Used

- Reading with colon separators
- Zero-padding with %02d
- Format conversion logic
- Conditional display (AM/PM)

Case Study 4: Currency Formatter

```
1 #include <stdio.h>
2 int main(void) {
3     double amount;
4     printf("Enter amount: ");
5     scanf("%lf", &amount);
6     printf("\nFormatting Options:\n");
7     printf("Basic:           Rs. %.2f\n", amount);
8     printf("With commas:      Rs. %'.2f\n", amount);
9     printf("Right-aligned:   Rs. %12.2f\n", amount);
10    printf("Zero-padded:     Rs. %012.2f\n", amount);
11    printf("Left-aligned:    Rs. %12.2f\n", amount);
12    printf("Scientific:      Rs. %e\n", amount);
13 }
```

Case Study 4: Sample Run

Input: 12345.67

Output:

Formatting Options:

Basic:	Rs. 12345.67
With commas:	Rs. 12,345.67
Right-aligned:	Rs. 12345.67
Zero-padded:	Rs. 00012345.67
Left-aligned:	Rs. 12345.67
Scientific:	Rs. 1.234567e+04

Note

The comma formatting (' flag) is locale-dependent and may not work on all systems.

Chapter Summary - Part 1

What We Learned

1. **Streams:** Sequences of bytes for I/O operations
2. **printf:** Formatted output with conversion specifiers
3. **Integer Formats:** d, i, o, u, x, X
4. **Floating-Point:** e, E, f, g, G with precision
5. **Characters/Strings:** c and s specifiers
6. **Field Widths:** Control output alignment and spacing
7. **Precision:** Different meanings for different types
8. **Format Flags:** -, +, space, #, 0
9. **Escape Sequences:** Special characters in output
10. **scanf:** Reading formatted input safely
11. **Other Specifiers:** p (pointers), %% (percent)

Questions?

Thank you for your attention! **Happy Coding!** 